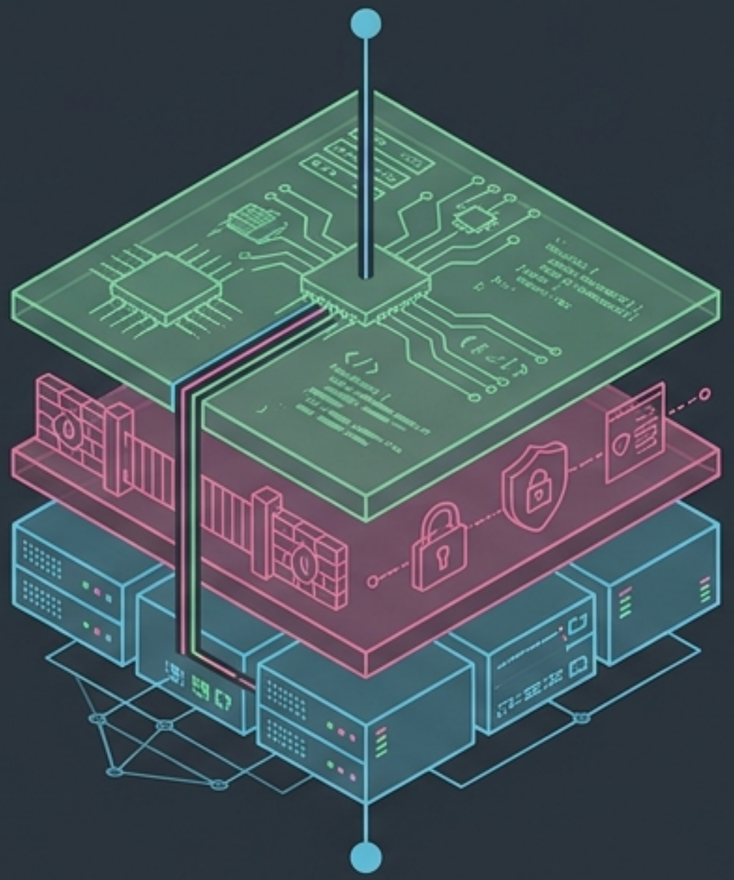


StockDesk

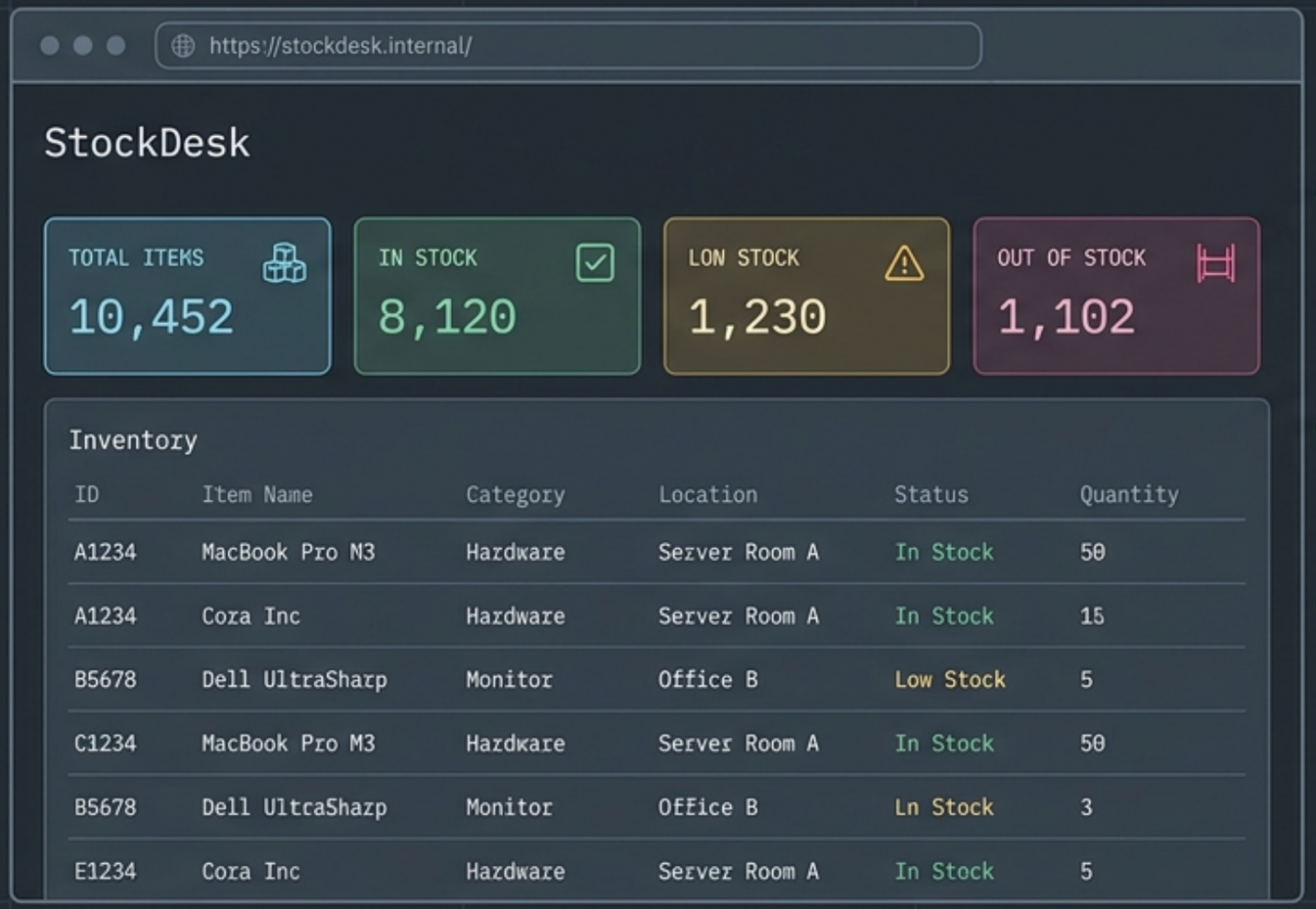
End-to-End Cloud-Native DevOps Capstone

1 App. 15+ Technologies. 1 Fully Automated Lifecycle.

Gemini Notebook

The StockDesk Application Vehicle

A hardware inventory tracker purpose-built to execute a production-grade delivery lifecycle.



FRONTEND LAYER

React 18 + Vite

(Responsive single-page inventory dashboard)

BACKEND LAYER

FastAPI / Python 3.12

(Async REST API with native OpenAPI/Swagger)

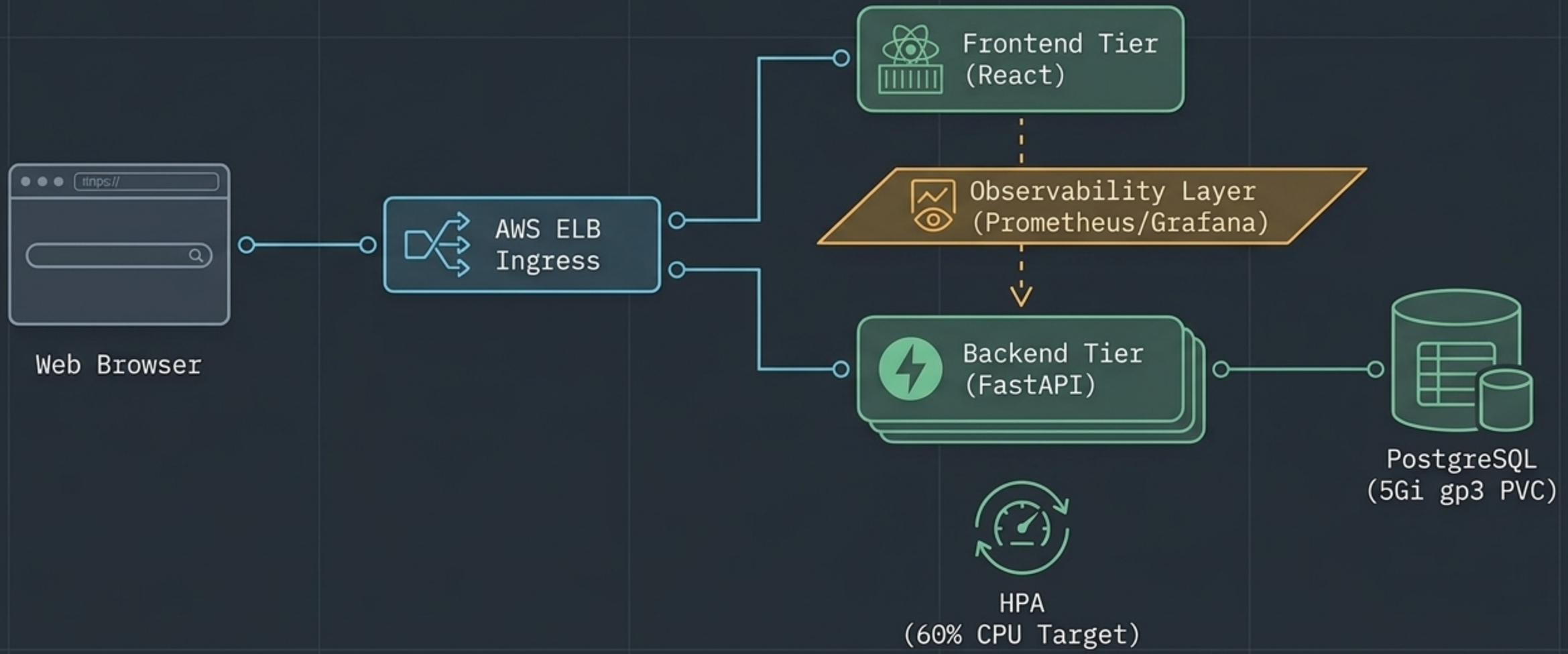
DATA LAYER

PostgreSQL 16 with Alembic

(Schema modeling and versioned migrations)

Gemini Notebook

Cloud-Native Architecture Blueprint

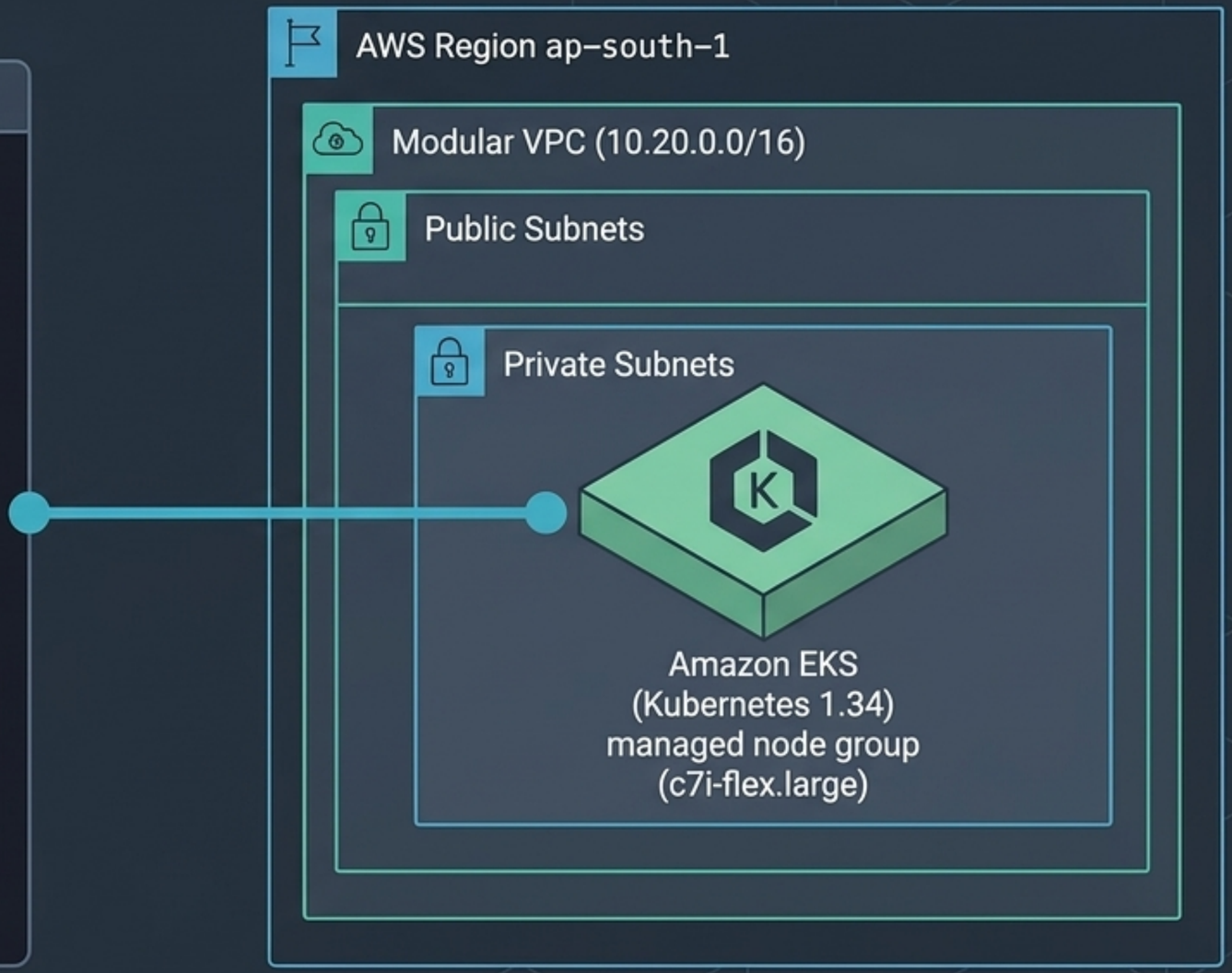


Gemini Notebook

Provisioning the Cloud Foundation

```
$ terraform apply
```

Cost Conscious Architecture:
Automated terraform destroy capabilities completely tear down the cluster and NAT gateways to prevent runaway billing (cap at ~$0.20/hr during evaluation).



Gemini Notebook

The CI/CD Automation Engine



Unit Testing:

Pytest execution with HTTPX (11/11 tests passing across logic, stats, and metrics).

Artifact Generation:

Multi-stage Docker container builds.

Immutable Registry:

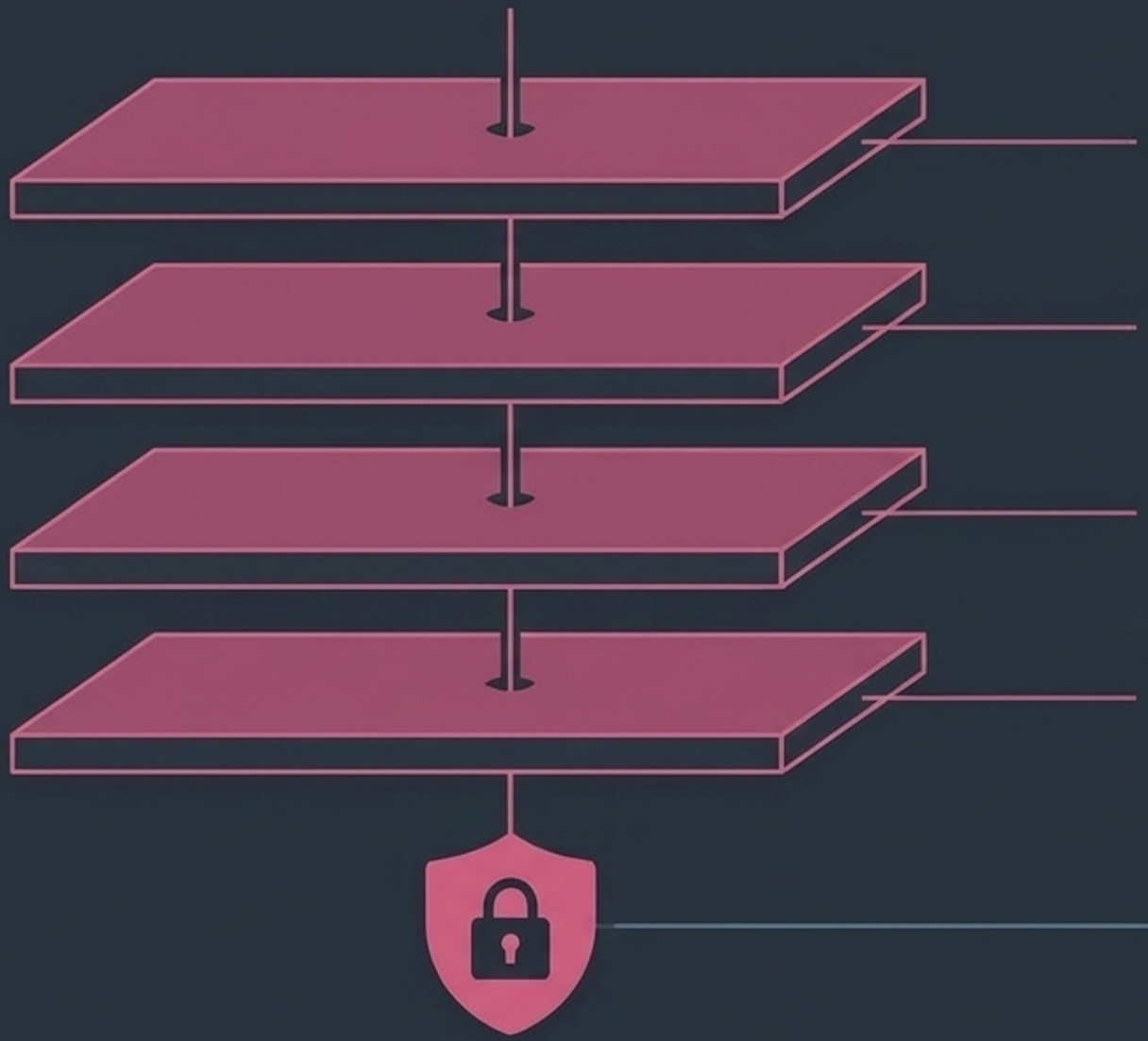
Push to GitHub Container Registry (GHCR).

Traceability:

Every container image is strictly tagged with its exact Git commit SHA, eliminating fragile latest tags.

Gemini Notebook

DevSecOps and the 4-Layer Security Gauntlet



Layer 1: **Secrets Scanning.**
`Gitleaks` (`.gitleaks.toml`) blocks leaked AWS keys or DB tokens.

Layer 2: **Static Analysis (SAST).**
`Bandit` flags unsafe shell executions or SQL injections in Python.

Layer 3: **Dependency Audit (SCA).**
`pip-audit` and `npm audit` cross-reference libraries against the NVD/GitHub Advisory DB.

Layer 4: **Container CVE Scan.**
Aqua `Trivy` scans the final image OS and layers.

```
The pipeline forcibly exits non-zero on any
unfixed HIGH or CRITICAL CVE.
```

Gemini Notebook

Hardening the Container Artifacts

	Standard Container	StockDesk Container
User Context	Runs as Root (UID 0)	**Strict Non-root execution** (Backend: appuser UID 10001, Frontend: nginx-unprivileged UID 101)
Build Footprint	Fat image with compilers	**Multi-stage builds stripping all toolchains from the runtime layer** (.dockerignore enforced)
Image Tagging	Overwrites latest	**Strict Git SHA traceability**
Vulnerability Profile	Unchecked OS flaws	**44 HIGH / 0 CRITICAL** (all unfixable upstream OS CVEs conditionally ignored; 0 Python vulnerabilities) ✅



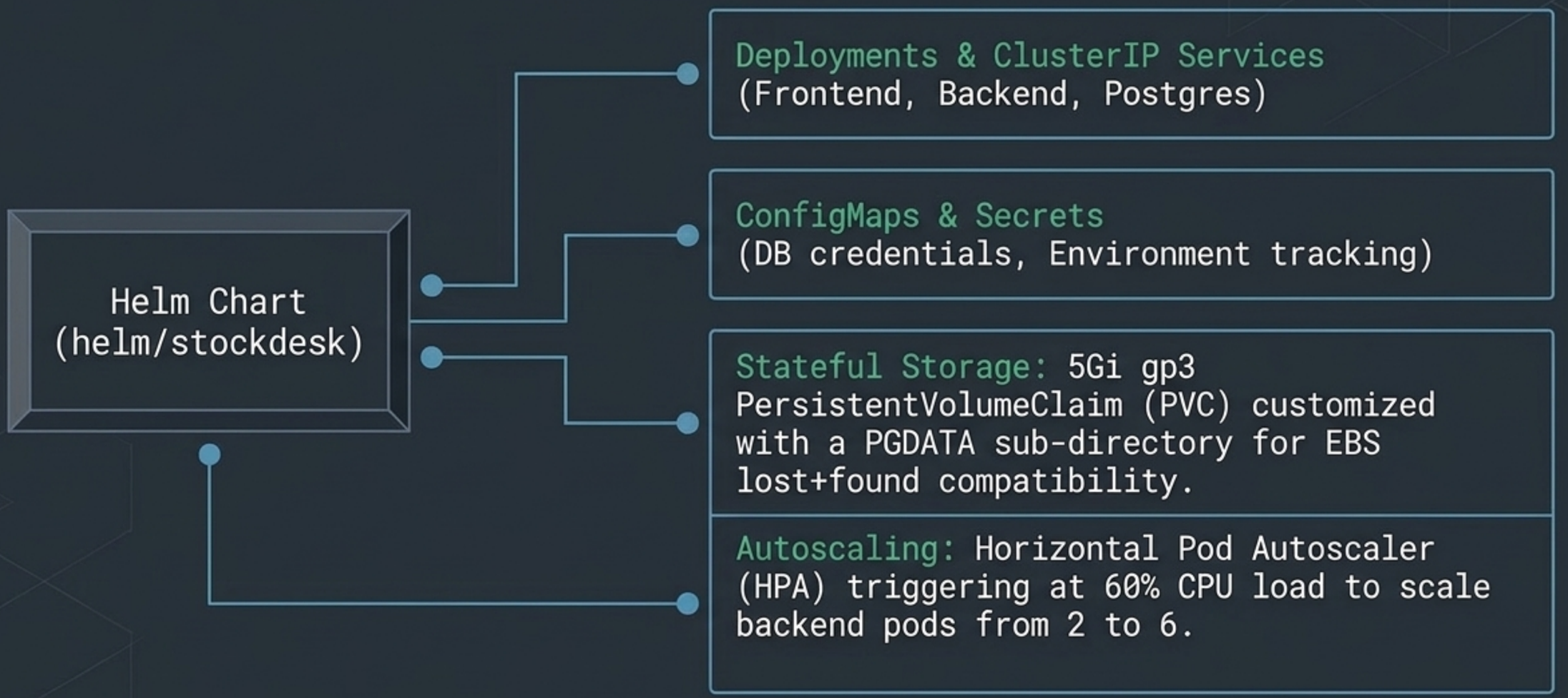




Gemini Notebook

Declarative State and Helm Packaging

Replacing raw YAML files with parameterized Helm templates for environment-specific execution (`values-dev.yaml` vs `values-prod.yaml`).



Gemini Notebook

Continuous Delivery via the GitOps Paradigm

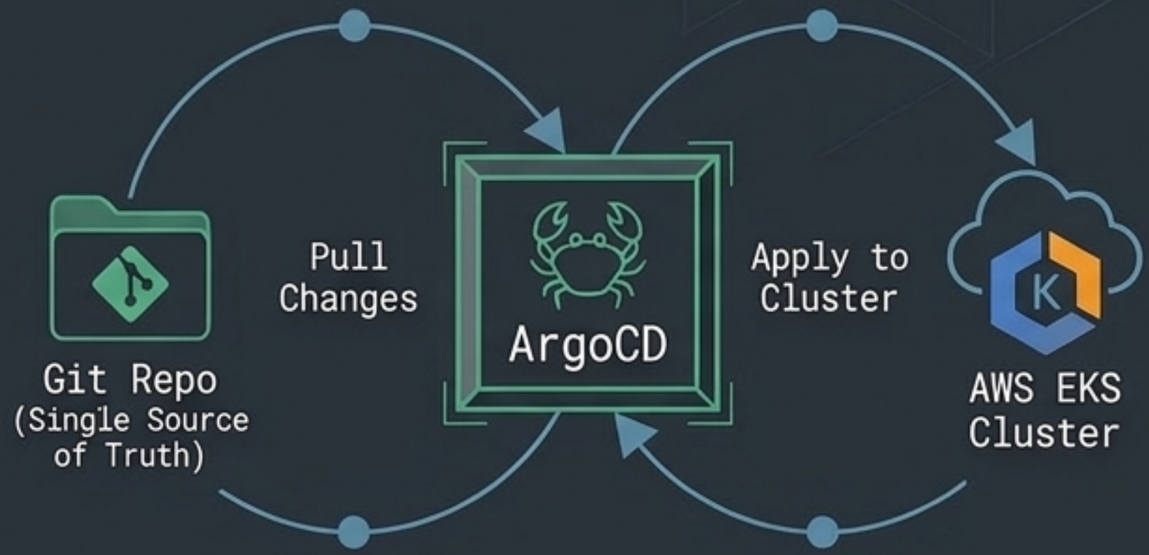
The Old Way (Push)

The Old Way (Push)



Fragile, requires cluster credentials in CI.

The GitOps Paradigm (Pull)



- Engine: ArgoCD continuously watches the GitHub repository and reconciles the AWS EKS cluster to the declared state.
- Self-Healing: Any manual kubectl drift injected into the cluster is instantly detected and overwritten by ArgoCD to match Git.

Gemini Notebook

Observability and Self-Healing



The Telemetry Flow

Instrumentation

FastAPI instrumentator exposes /metrics.

Scraping

Prometheus ServiceMonitor scrapes target pods every 15 seconds.

Visualization

Grafana dashboards track Requests/sec, p95 latency, and 5xx error rates.

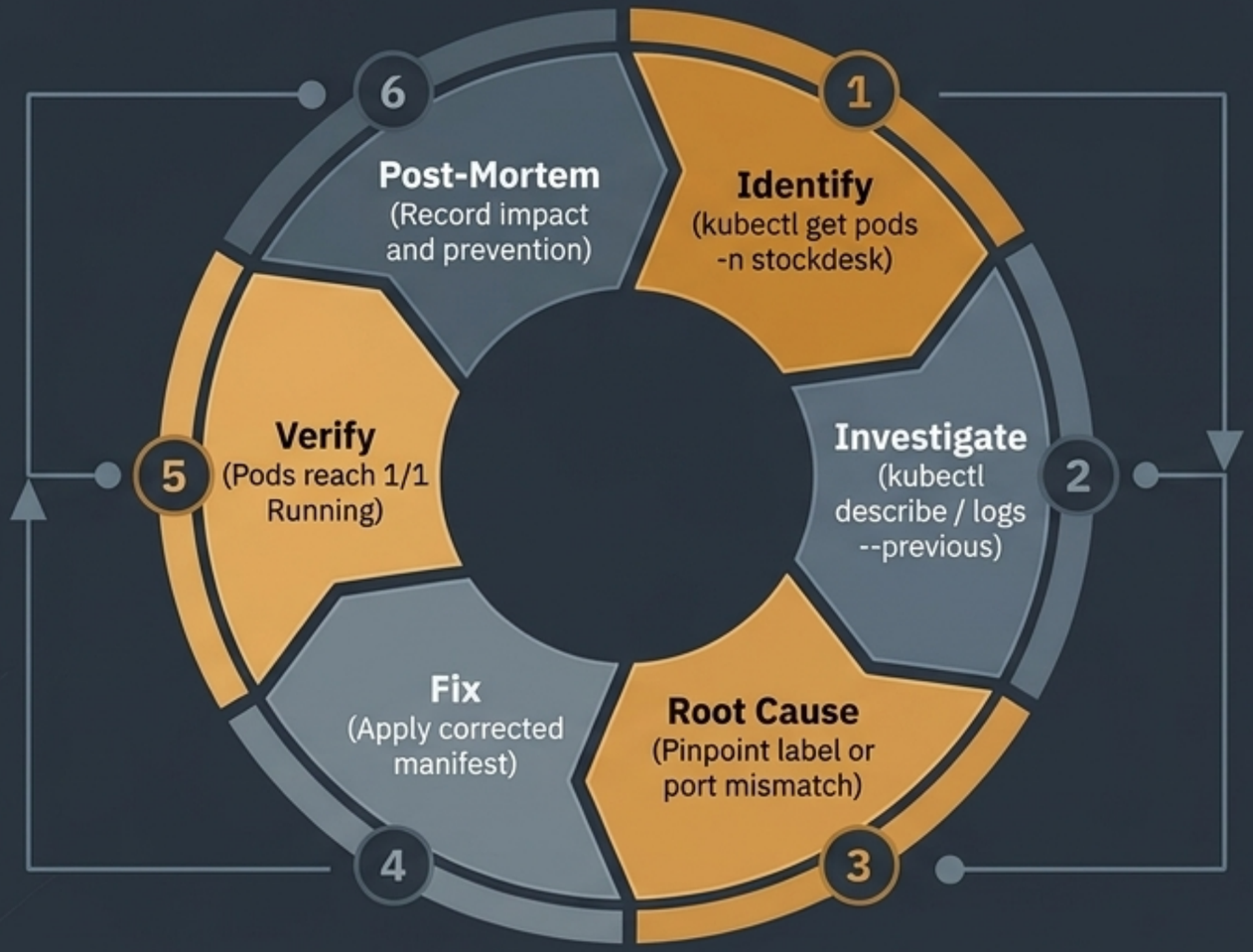
Alerting

Automated alerts triggered via monitoring/alert-rules.yaml if backend is down for >1 minute or if 5xx error rates exceed 5% of total traffic.

Gemini Notebook

Resilience Engineering and the Troubleshooting Lab

Demonstrating true SRE maturity by injecting deliberate chaos (e.g., `ImagePullBackOff`, Service Selector mismatches) to execute structured incident response.



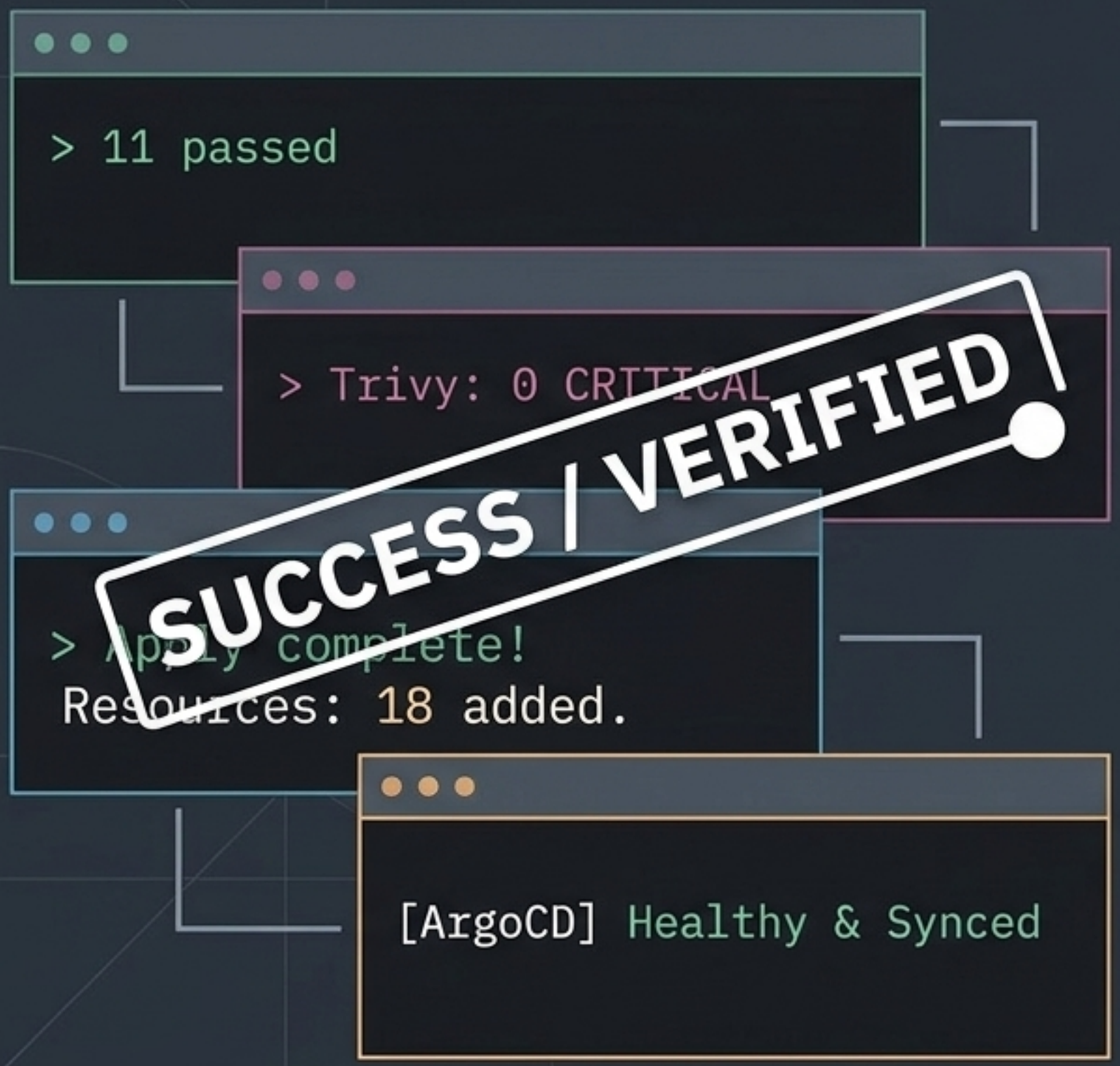
Gemini Notebook

The DevOps Capability Matrix

Phase	Tool	Outcome
Code	Git / GitHub	Single Source of Truth & Versioning
Build	Docker / GitHub Actions	Multi-stage, Immutable SHA Artifacts
Secure	Bandit / Gitleaks / Trivy	Zero-Trust 4-Layer Security Gate
Deploy	ArgoCD / Helm	Declarative GitOps Reconciliation
Operate	Terraform / AWS EKS	Scalable, Reproducible Infrastructure
Observe	Prometheus / Grafana	Real-time Telemetry & Alerting

Gemini Notebook

Proof of Execution and Live Deployment



Infrastructure is ephemeral; Git state is permanent.

Security must be automated to be effective.

Full-stack visibility enables true self-healing.

Scan or visit the link to watch the full live deployment walkthrough from code commit to cluster update.

[QR Code / Link]

Gemini Notebook